WEEK 1 · DAY 4 · OVERVIEW

Yield From Price



Created by **Jeff Lenamon**

ABOUT 45 MIN WITH EXERCISES / 9 SECTIONS / 4 EXERCISES

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The Desk Lead's Note

Dealers quote the four new bonds on price this morning: Quorvane 99.125, Dunmarrow 100.5, Pellwick 100.875, Halvering 97.75. Notebook 02 went from yield to price; I need the other way. What yield does each quote imply? Show me how Excel gets there and how close it lands. Put the search in bondmath, docstring first, prove it against Excel's RATE and YIELD, and make it refuse a price no yield explains. If an edit breaks it, show me you can undo it, then take it home.

ABOUT 45 MIN WITH EXERCISES / 9 SECTIONS / 4 EXERCISES

Your Route Through Day 4

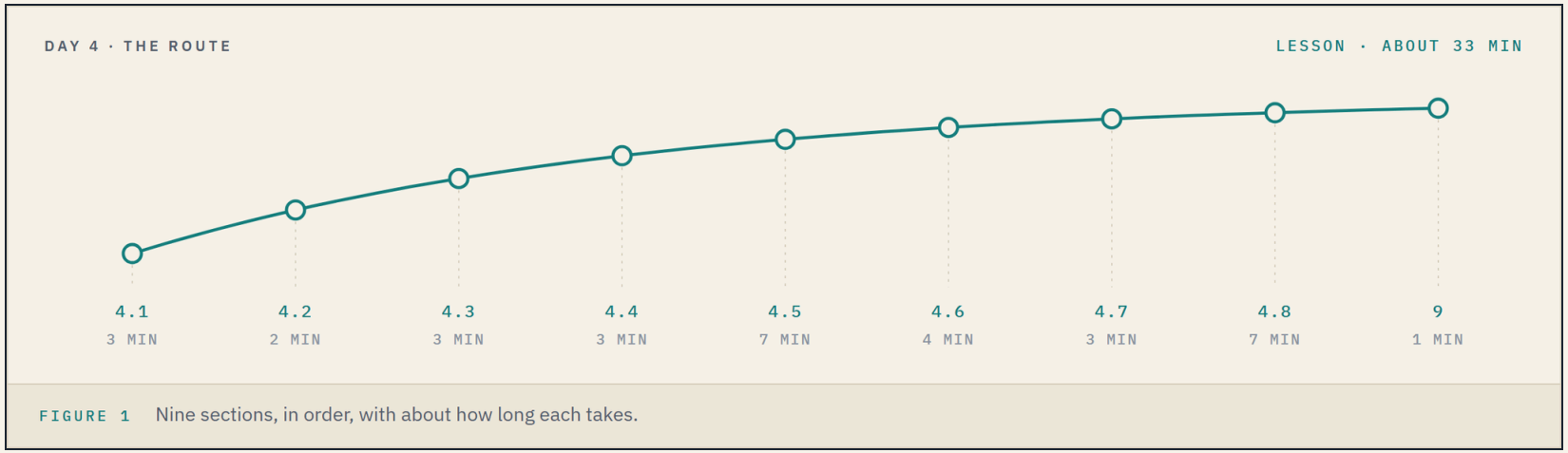


FIGURE 1 Nine sections, in order, with about how long each takes.

	SECTION	ABOUT
4.1	Why Yield Has No Formula	3 min
4.2	Goal Seek in Excel	2 min
4.3	Bisection by Hand in a Table	3 min
4.4	A `while` Loop That Halves the Bracket	3 min
4.5	`yield_from_price` With a Check That Stops	7 min

	SECTION	ABOUT
4.6	`pytest.raises`	4 min
4.7	Price to Yield to Price, Round Trip	3 min
4.8	The Undo Button	7 min
9	Bring It Home	1 min

Time. About 33 minutes for the lesson, then about 12 for the four exercises.

4.1 Why Yield Has No Formula

Yield to price is arithmetic; price to yield has no formula

Quorvane, 5.25 coupon, 5 years: ten half-year payments, y in every one

k = 1	k = 2	k = 3	k = 4	k = 5	k = 6	k = 7	k = 8	k = 9	k = 10
2.625 / (1+y/200)**1	2.625 / (1+y/200)**2	2.625 / (1+y/200)**3	2.625 / (1+y/200)**4	2.625 / (1+y/200)**5	2.625 / (1+y/200)**6	2.625 / (1+y/200)**7	2.625 / (1+y/200)**8	2.625 / (1+y/200)**9	102.625 / (1+y/200)**10

2.625 = 5.25 / 2, the coupon a half-year; y / 200 = y / 100 / 2, the yield a half-year, as a decimal

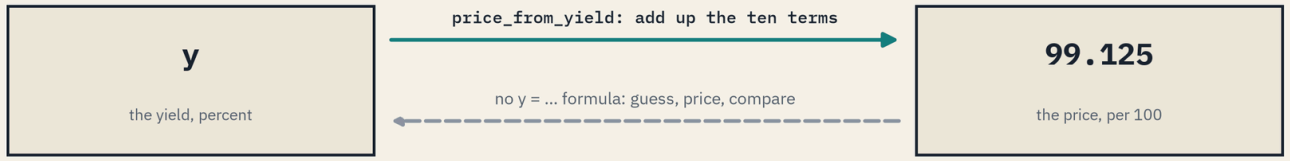


FIGURE 2 Quorvane's price is ten terms that all hold the yield y, so there is no y = formula.

FIVE GUESSES

```
guesses = [5.0, 6.0, 5.5, 5.4, 5.45]
for guess_pct in guesses:
```

```
5.00 percent: price 101.0940, above the quote, go higher
6.00 percent: price 96.8012, below the quote, go lower
5.50 percent: price 98.9200, below the quote, go lower
```

Every guess says which way to go, because price falls as yield rises: that one fact is all a search needs.

4.2 Goal Seek in Excel

EXCEL SIDE BY SIDE

JOB	EXCEL	RESULT
A first guess	**5** in A1	
The price at the guess	**=PRICE(DATE(2026,9,30),DATE(2031,9,30),5.25/100,A1/100,100,2,0)** in B1	
Guess until the price is the quote	Data, What-If Analysis, Goal Seek: Set cell **B1**, To value **99.125**, By changing cell **A1**, then OK	A1 5.452248461259856, B1 99.12520671813279

WHERE GOAL SEEK STOPPED

	TICKER	YIELD_PCT	PRICE_GAP	EXCEL_YIELD_PCT	YIELD_GAP_BP
0	QVNE	5.452248	0.000207	5.452297	-0.004804
1	DNMR	6.036364	0.000001	6.036364	-0.000015
2	PLWK	4.435154	0.000356	4.435281	-0.012744
3	HLVF	7.828571	0.000262	7.828610	-0.003889

Where Goal Seek stops depends on where it starts: an answer key needs a search that stops where we say, every time.

4.3 Bisection by Hand in a Table

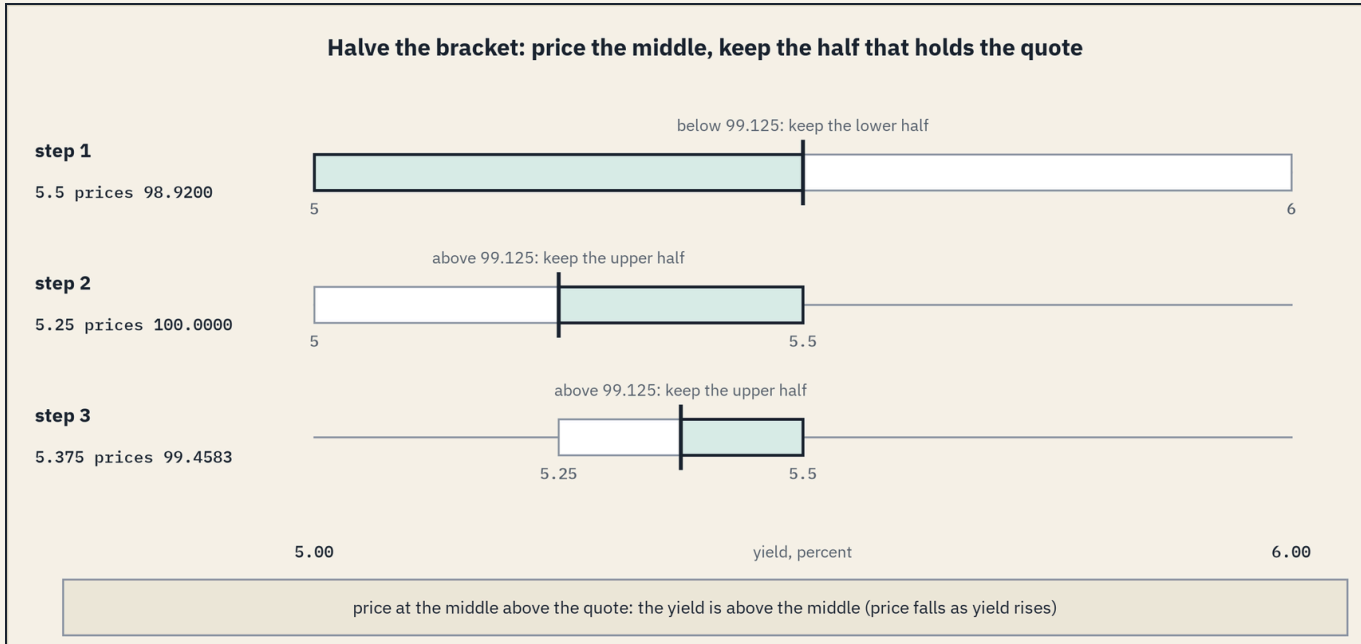


FIGURE 3 Three halvings of 5 to 6 percent against the 99.125 quote leave the yield between 5.375 and 5.5.

THE FIRST HALVINGS

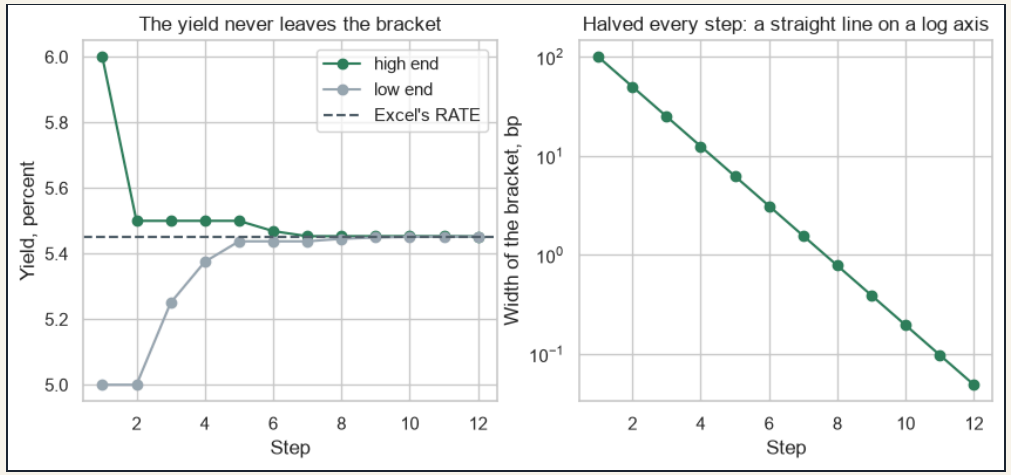
STEP	LOW_PCT	HIGH_PCT	MID_PCT
1	5.000000	6.000000	5.500000
2	5.000000	5.500000	5.250000
3	5.250000	5.500000	5.375000
4	5.375000	5.500000	5.437500
5	5.437500	5.500000	5.468750

The width halves every row, from 100 bp to 0.0488; the twelfth middle is 5.452393.

Watching the Bracket Close

EXCEL SIDE BY SIDE

JOB	EXCEL	RESULT
The middle, priced	`=(A2+B2)/2` in C2, `=PRICE(DATE(2026,9,30),DATE(2031,9,30),5.25/100,C2/100,100,2,0)` in D2	
Keep a half	`=IF(D2>99.125,C2,A2)` in A3, `=IF(D2>99.125,B2,C2)` in B3, C2:D2 copied to row 3, then A3:D3 filled down to row 31	C13 5.452392578125, C31 5.452296503819525



THE LAST HALVINGS

STEP	MID_PCT	WIDTH_BP
10	5.452148	0.1953
11	5.452637	0.0977
12	5.452393	0.0488

The bracket halves every step and the yield never leaves it: bisection cannot miss.

4.4 A while Loop That Halves the Bracket

HALVE UNTIL IT IS NARROW

```
low_pct, high_pct = 5.0, 6.0
while high_pct - low_pct > 1e-10:
    mid_pct = (low_pct + high_pct) / 2
    if bondmath.price_from_yield(5.25, mid_pct, 5) > 99.125:
        low_pct = mid_pct
    else:
        high_pct = mid_pct
    steps += 1
print(f'{steps} steps: Quorvane at 99.125 yields {quorvane_yield_pct:.8f} percent')
```

```
34 steps: Quorvane at 99.125 yields 5.45229650 percent
```

AGAINST EXCEL'S RATE

```
Excel's RATE: 5.45229650 percent, a gap of 2.0e-08 bp
```

Your loop lands on Excel's `RATE`: 5.45229650 percent, 2.0e-08 bp apart.

4.5 yield_from_price With a Check That Stops

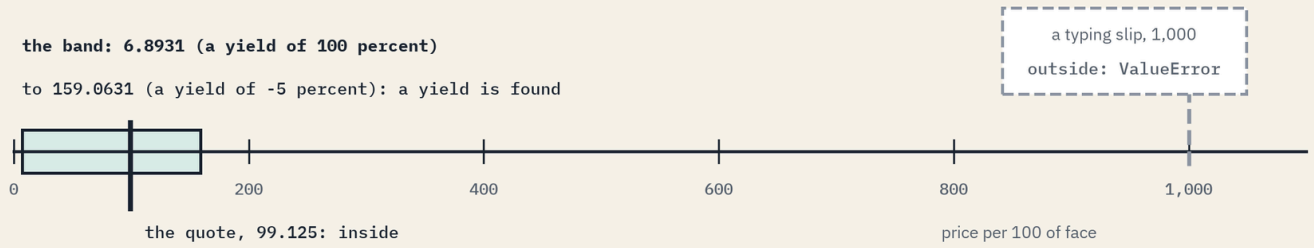


FIGURE 4 Any yield from -5 to 100 percent prices between 6.8931 and 159.0631; a quote of 1,000 is refused.

FOUR QUOTES, THREE ANSWERS

	TICKER	YIELD_PCT	EXCEL_RATE_PCT
0	QVNE	5.452297	5.452297
1	DNMR	6.036364	6.036364
2	PLWK	4.435281	4.435281
3	HLVF	7.828610	7.828610

Three columns, one answer to the sixth decimal.

The Function and Excel

THE FUNCTION

```
def yield_from_price(coupon_pct: float, price: float, years: float, frequency: int = 2) -> float:
    highest_price = price_from_yield(coupon_pct, low_pct, years, frequency)
    lowest_price = price_from_yield(coupon_pct, high_pct, years, frequency)
    if not lowest_price <= price <= highest_price:
    while high_pct - low_pct > 1e-10:
    return (low_pct + high_pct) / 2
```

EXCEL SIDE BY SIDE

JOB	EXCEL	RESULT
Yield from price, from periods	`=RATE(2*5,5.25/2,-99.125,100)*2`	0.05452296503527843
Yield from price, from dates	`=YIELD(DATE(2026,9,30),DATE(2031,9,30),5.25/100,99.125,100,2,0)`	0.054522965032957
A price of 1,000	`=YIELD(DATE(2026,9,30),DATE(2031,9,30),5.25/100,1000,100,2,0)*100`	-39.3451

Four quotes, four yields that match Excel, and a price no yield in the bracket explains is refused.

4.6 pytest.raises

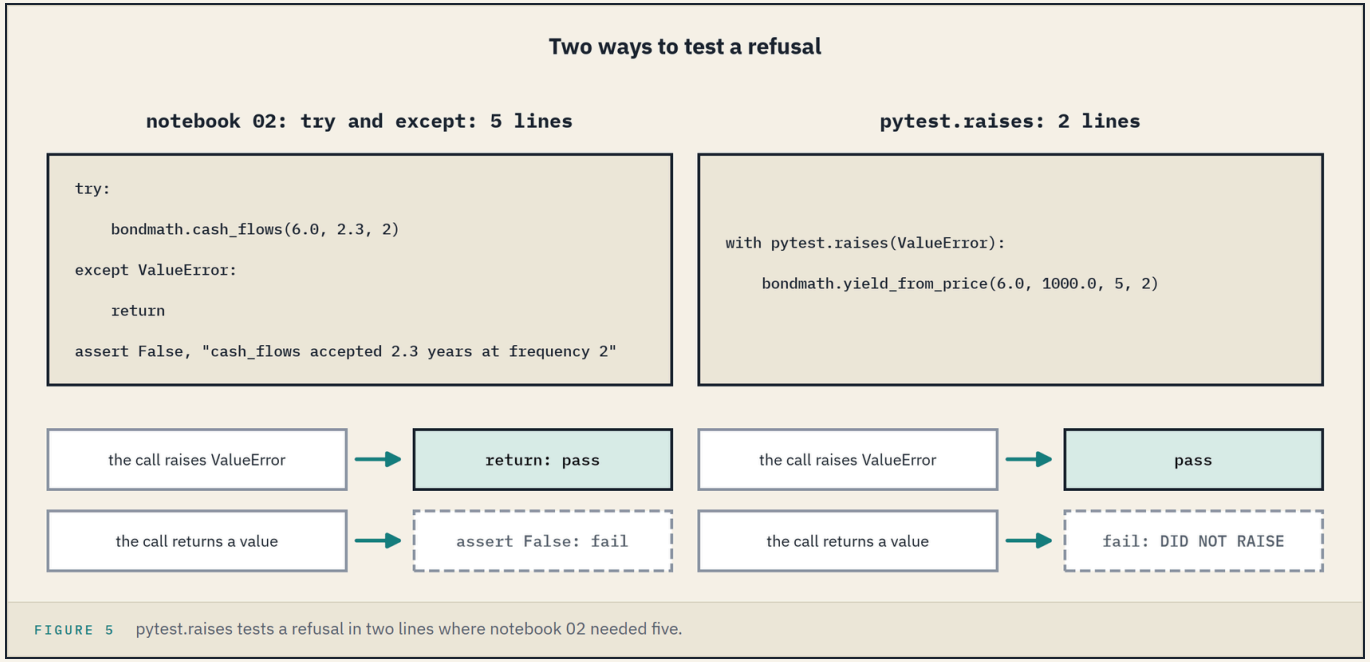


FIGURE 5 pytest.raises tests a refusal in two lines where notebook 02 needed five.

CATCH THE ERROR

```
with pytest.raises(ValueError) as caught:
    bondmath.yield_from_price(5.25, 1000.0, 5)
print('Caught:', caught.value)
```

WHEN NOTHING IS RAISED

```
with pytest.raises(ValueError):
    bondmath.yield_from_price(5.25, 99.125, 5)
```

```
Failed: DID NOT RAISE ValueError
```

10 passed: **the module matches Excel on 35 rows, and a test now fails if the refusal ever goes.**

4.7 Price to Yield to Price, Round Trip

YIELD, PRICE, YIELD

TICKER	ISSUE_YIELD_PCT	PRICE	GAP_BP
QVNE	5.400	99.350327	+1.1e-09
DNMR	6.125	100.000000	-1.9e-09
PLWK	4.600	100.415887	-1.6e-09
HLVF	7.700	98.622736	+1.6e-09

QUOTE, YIELD, QUOTE

TICKER	QUOTE	PRICE_BACK	GAP
QVNE	99.125	99.125000000	+1.4e-11
DNMR	100.500	100.500000000	+2.5e-10
PLWK	100.875	100.875000000	+5.2e-11
HLVF	97.750	97.750000000	-1.2e-10

THE ROUND-TRIP TEST

```
def test_yield_round_trip():
    bond_price = bondmath.price_from_yield(5.25, 6.1, 7, 2)
    assert bondmath.yield_from_price(5.25, bond_price, 7, 2) == pytest.approx(6.1, abs=1e-8)
```

Two functions that undo each other to nine decimals: the round trip proves they agree, Excel's rows that they are right.

4.8 The Undo Button

WHAT CHANGED

```
!git -C "{work}" status --short
!git -C "{work}" diff --stat
```

```
M bondmath.py
 M test_bondmath.py
?? __pycache__/
 bondmath.py      | 29 ++++++++++++++++++++++++++++++
 test_bondmath.py | 30 ++++++++++++++++++++++++++++++
 2 files changed, 59 insertions(+)
```

PUT IT BACK

```
!git -C "{work}" restore bondmath.py
!git -C "{work}" status --short
```

```
?? __pycache__/
```

EXCEL SIDE BY SIDE

The nearest move in a workbook is to close it without saving: you are back at the last save.

> **`git restore` throws the change away for good, so read the diff first.**

Bring It Home

BRING IT HOME

```
$work = "$env:TEMP\pfi_course2_04_k4w7q2xr"
Set-Location "$HOME\projects\my_bondmath"
Copy-Item "$work\bondmath.py", "$work\test_bondmath.py" .
python -m pytest -q
git add bondmath.py test_bondmath.py
git log --oneline
```

THE COMMITS IT HOLDS

```
!git -C "{work}" log --oneline
```

```
07294a2 Notebook 04: yield_from_price
3f57cac Notebook 03: convert_yield
408a678 Notebook 02: cash_flows and price_from_yield
d029474 Notebook 01: future_value and present_value
```

A COMMIT, AS THE WORK FOLDER MADE IT

```
!git -C "{work}" add bondmath.py test_bondmath.py
!git -C "{work}" commit -m "{subject}" -m "{body}"
```

```
[main 07294a2] Notebook 04: yield_from_price
 2 files changed, 59 insertions(+)
```

my_bondmath **holds six functions, eleven tests, and four commits.**

You Can Now

- Say why a yield has no formula, and why price falling as yield rises is all a search needs.
- Find a yield with Excel's Goal Seek, and say why its answer depends on where it starts.
- Halve a bracket by hand in Python and in a sheet, and say why bisection cannot miss.
- Write a **`while`** loop that stops when the bracket is narrow enough.
- Solve a yield from a price with **`yield_from_price`**, and in Excel with **`RATE`** and **`YIELD`**.
- Test a refusal with **`pytest.raises`**, and two functions against each other with a round trip.
- Undo a bad edit with **`git status`**, **`git diff`**, and **`git restore`**.

NEXT
WEEK 1 · DAY 5

Case Study: A New Issue Calendar

Eight new issues described with the week's two pricing functions, every number tied to Excel, and sent as a workbook Excel calculates itself.